

Lab 6 : Empirical Analysis of Quick Sort

Theory:

Quick Sort is a highly efficient sorting algorithm that, much like Merge Sort, is based on the "Divide and Conquer" approach. It works by selecting a "pivot" element from the array and partitioning the other elements into two sub-arrays according to whether they are less than or greater than the pivot.

The sub-arrays are then sorted recursively. This in-place partitioning makes Quick Sort extremely fast in practice, achieving an average Time Complexity of $O(n \log n)$. However, unlike Merge Sort, if a bad pivot is consistently chosen (like in an already sorted array), its performance can degrade to a worst-case scenario of $O(n^2)$, though this is rare with proper pivot selection.

Algorithm

```
int partition(vector<int>& arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSort(vector<int>& arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
```

Expressing Complexity of cases

Case	Time Complexity	Condition	Comparisons	Swaps
Best	$O(n \log n)$	Pivot always splits evenly	$n \log n$	$n \log n$
Average	$O(n \log n)$	Random order / Random pivot	$\sim 1.39 n \log n$	$\sim 0.69 n \log n$
Worst	$O(n^2)$	Already sorted / Reverse sorted	$n(n - 1) / 2$	$n(n - 1) / 2$

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

long long count = 0;

void generate(int *arr, int n) {
    for (int i = 0; i < n; i++) { arr[i] = rand() % 1000; count++; }
}

void generateINC(int *arr, int n) {
    arr[0] = rand() % 1000; count++;
    for (int i = 1; i < n; i++) { arr[i] = arr[i - 1] + (rand() % 50 + 1); count += 2; }
}

void generateDEC(int *arr, int n) {
    arr[0] = rand() % 5000; count++;
    for (int i = 1; i < n; i++) { arr[i] = arr[i - 1] - (rand() % 50 + 1); count += 2; }
}

void swap(int *x, int *y) {
    int temp = *x; *x = *y; *y = temp; count += 3;
}

int partition(int *arr, int l, int r) {
    int p = arr[r], i = l - 1; count += 2;
    for (int j = l; j < r; j++) {
        if (arr[j] < p) { swap(&arr[++i], &arr[j]); }
        count++;
    }
    swap(&arr[i + 1], &arr[r]);
    return i + 1;
}

void quicksort(int *arr, int l, int r) {
    if (l < r) {
        int p = partition(arr, l, r); count++;
        quicksort(arr, l, p - 1);
        quicksort(arr, p + 1, r);
    }
}

void print() { printf("Count:: %lld\n", count); }
```

```

int main() {
    srand(time(NULL));
    for (int n = 1000; n <= 10000; n += 1000) {
        printf("\n023bscit009_binesh\nFor input n:: %d", n);
        int *a = (int *)malloc(sizeof(int) * n);
        int *b = (int *)malloc(sizeof(int) * n);
        int *c = (int *)malloc(sizeof(int) * n);

        printf("\nAscending:: \n"); count = 0;
        generateINC(a, n); quicksort(a, 0, n - 1); print();

        printf("Random:: \n"); count = 0;
        generate(b, n); quicksort(b, 0, n - 1); print();

        printf("Descending:: \n"); count = 0;
        generateDEC(c, n); quicksort(c, 0, n - 1); print();

        free(a); free(b); free(c);
    }
    return 0;
}

```

Observations/Outputs:

Output 1 :

```
023bscit009_binesh
For input n:: 1000
Ascending::
Count:: 2005993
Random::
Count:: 33251
Descending::
Count:: 1255993
```

Output 2 :

```
023bscit009_binesh
For input n:: 5000
Ascending::
Count:: 50029993
Random::
Count:: 204216
Descending::
Count:: 31279993
```

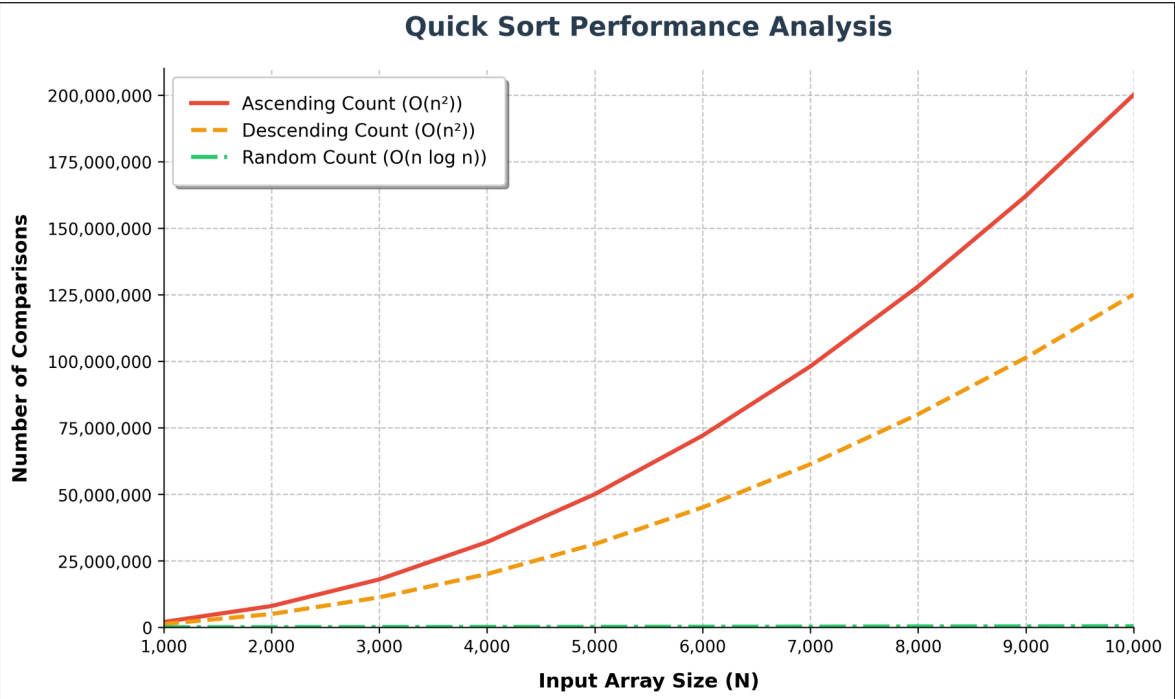
Output 3 :

```
023bscit009_binesh
For input n:: 10000
Ascending::
Count:: 200059993
Random::
Count:: 413668
Descending::
Count:: 125059993
```

Data Table:

Input Size	Ascending	Random	Descending
1,000	2,005,993	30,594	1,255,993
2,000	8,011,993	65,494	5,011,993
3,000	18,017,993	103,920	11,267,993
4,000	32,023,993	145,433	20,023,993
5,000	50,029,993	199,250	31,279,993
6,000	72,035,993	240,598	45,035,993
7,000	98,041,993	269,633	61,291,993
8,000	128,047,993	335,924	80,047,993
9,000	162,053,993	393,227	101,303,993
10,000	200,059,993	428,693	125,059,993

Function Growth:



Conclusion:

The results of this lab clearly demonstrate how heavily Quick Sort depends on the choice of the pivot element. When dealing with randomized data, Quick Sort performs incredibly well, staying close to its theoretical $O(n \log n)$ efficiency—even at an input size of 10,000, it only took around 425,000 operations.

However, things change drastically when the array is already sorted (Ascending) or reverse sorted (Descending). Because our algorithm blindly picks the last element as the pivot, an already ordered list forces it to partition the array one single element at a time. We can clearly see this in the data, where the operations shot up to over 200 million for the ascending case at $n=10,000$. This confirms its $O(n^2)$ worst-case behavior. Overall, this proves that while Quick Sort is extremely fast for random data, it requires better pivot selection methods (like choosing a random pivot or the median) to avoid completely breaking down on sorted lists in the real world.